

```

import numpy as np
import keras
from keras.datasets import mnist
from keras.models import Sequential
from keras.layers import Conv2D, MaxPool2D, Dense, Flatten

(X_train, Y_train), (X_test, Y_test)= mnist.load_data()

print(f"Original X_train shape: {X_train.shape}")

X_train = np.array(X_train)
Y_train = np.array(Y_train)
X_test = np.array(X_test)
Y_test = np.array(Y_test)

X_train = X_train.astype('float32') / 255
X_test = X_test.astype('float32') / 255

X_train = np.expand_dims(X_train, axis=-1)
X_test = np.expand_dims(X_test, axis=-1)

Y_train = keras.utils.to_categorical(Y_train, 10)
Y_test = keras.utils.to_categorical(Y_test, 10)
print('Training data:')
print(Y_train.shape)
print(X_train.shape)
print('-----')
print('Testing Data:')
print(X_test.shape)
print(Y_test.shape)

#Model

model = Sequential()
model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPool2D((3,3)))
model.add(Conv2D(32, (3,3), activation='relu'))
model.add(MaxPool2D((3,3)))
model.add(Flatten())
model.add(Dense(64, activation='relu'))
model.add(Dense(10, activation='softmax'))
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])

model.fit(X_train, Y_train, epochs=20)

pred = model.predict(X_test[0:3])
print(pred)
print(Y_test[3])

```

Original X_train shape: (60000, 28, 28)

Training data:

(60000, 10)

(60000, 28, 28, 1)

Testing Data:

(10000, 28, 28, 1)

(10000, 10)

Epoch 1/20

1875/1875 ————— 6s 3ms/step - accuracy: 0.9327 - loss: 0.2242

Epoch 2/20

1875/1875 ————— 5s 3ms/step - accuracy: 0.9771 - loss: 0.0724

Epoch 3/20

1875/1875 ————— 5s 3ms/step - accuracy: 0.9832 - loss: 0.0531

Epoch 4/20

1875/1875 ————— 5s 3ms/step - accuracy: 0.9866 - loss: 0.0422

Epoch 5/20

1875/1875 ————— 5s 3ms/step - accuracy: 0.9889 - loss: 0.0348

Epoch 6/20

1875/1875 ————— 5s 3ms/step - accuracy: 0.9900 - loss: 0.0306

Epoch 7/20

1875/1875 ————— 5s 3ms/step - accuracy: 0.9913 - loss: 0.0265

Epoch 8/20

1875/1875 ————— 5s 3ms/step - accuracy: 0.9928 - loss: 0.0229

Epoch 9/20

1875/1875 ————— 5s 3ms/step - accuracy: 0.9935 - loss: 0.0199

Epoch 10/20

1875/1875 ————— 5s 3ms/step - accuracy: 0.9942 - loss: 0.0178

Epoch 11/20

1875/1875 ————— 5s 3ms/step - accuracy: 0.9947 - loss: 0.0152

Epoch 12/20

1875/1875 ————— 5s 3ms/step - accuracy: 0.9955 - loss: 0.0139

Epoch 13/20

1875/1875 ————— 5s 3ms/step - accuracy: 0.9956 - loss: 0.0130

Epoch 14/20

1875/1875 ————— 5s 3ms/step - accuracy: 0.9962 - loss: 0.0114

```

Epoch 15/20
1875/1875 _____ 5s 3ms/step - accuracy: 0.9967 - loss:
0.0101
Epoch 16/20
1875/1875 _____ 5s 3ms/step - accuracy: 0.9965 - loss:
0.0101
Epoch 17/20
1875/1875 _____ 5s 3ms/step - accuracy: 0.9970 - loss:
0.0087
Epoch 18/20
1875/1875 _____ 5s 3ms/step - accuracy: 0.9971 - loss:
0.0081
Epoch 19/20
1875/1875 _____ 6s 3ms/step - accuracy: 0.9971 - loss:
0.0081
Epoch 20/20
1875/1875 _____ 5s 3ms/step - accuracy: 0.9976 - loss:
0.0072
1/1 _____ 0s 51ms/step
[[3.78689961e-11 3.70050635e-09 2.46979141e-07 4.70300038e-05
 7.23775351e-09 4.80677694e-12 3.80706237e-18 9.99945283e-01
 2.21506742e-08 7.37394566e-06]
 [3.94458908e-07 2.78133072e-12 9.99999642e-01 2.18122320e-08
 1.36905975e-16 1.02041744e-19 4.13930626e-11 7.72542752e-15
 2.50866911e-10 1.68702807e-14]
 [1.39527401e-09 9.99999642e-01 1.12962487e-10 1.65184654e-11
 5.64184219e-08 1.77624204e-09 2.69722311e-09 2.39700000e-07
 8.76343387e-10 7.49565210e-09]]
[1. 0. 0. 0. 0. 0. 0. 0. 0. 0.]

model = Sequential()
model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPool2D((3,3)))
model.add(Conv2D(32, (3,3), activation='tanh'))
model.add(MaxPool2D((3,3)))
model.add(Flatten())
model.add(Dense(64, activation='tanh'))
model.add(Dense(10, activation='sigmoid'))
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])

model.fit(X_train, Y_train, epochs=2)

pred = model.predict(X_test[0:3])
print(pred)
print(Y_test[3])

Epoch 1/2
1875/1875 _____ 6s 3ms/step - accuracy: 0.9465 - loss:
0.1907

```

```
Epoch 2/2
1875/1875 _____ 5s 3ms/step - accuracy: 0.9829 - loss:
0.0544
WARNING:tensorflow:5 out of the last 316 calls to <function
TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_dist
ributed at 0x0000024A0556E3E0> triggered tf.function retracing.
Tracing is expensive and the excessive number of tracings could be due
to (1) creating @tf.function repeatedly in a loop, (2) passing tensors
with different shapes, (3) passing Python objects instead of tensors.
For (1), please define your @tf.function outside of the loop. For (2),
@tf.function has reduce_retracing=True option that can avoid
unnecessary retracing. For (3), please refer to
https://www.tensorflow.org/guide/function#controlling\_retracing and
https://www.tensorflow.org/api\_docs/python/tf/function for more
details.
1/1 _____ 0s 56ms/step
[[2.1493445e-01 6.8338299e-01 8.2877922e-01 9.5752865e-01 6.1174440e-
03
 1.3586244e-01 2.8793063e-04 9.9994206e-01 3.4243864e-01 8.9047045e-
01]
 [9.8328328e-01 9.1294742e-01 9.9993831e-01 5.5214864e-01 2.0538399e-
02
 5.1829158e-03 2.2153106e-01 6.4604676e-01 2.6077574e-01 5.4406092e-
02]
 [2.3541875e-01 9.9999130e-01 1.6942090e-01 1.9530897e-01 7.8918010e-
01
 7.0852004e-02 7.8261036e-01 6.5854740e-01 3.1571221e-01 1.8356159e-
01]]
[1. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
```